Project 1 / NotesApp risk analysis

HAOJIE SONG / 2023905496

The required outputs are on slide 31: a data-flow diagram, STRIDE coverage, risk scores, and fixes for the top three risks.

Scope and assets

Code references resolve in the teacher's original commit 8d5916436b7516547898b17fc856a54807b3638f of the course application.

Protect student identities, grades, private comments, account credentials, session data, audit records, and service availability.

The teacher's Compose file publishes web port 8080 and DB port 5432 on all host interfaces. The web container runs as root and mounts the host Docker socket and the project tree.

The model includes student, teacher, admin, operator, browser, web, database, local files, the Ping target, the Docker host, and network DB clients.

Runtime DB checks used a loopback-only local overlay. The overlay changes ports, mounts, networks, and resource limits. It does not change DB settings.

Sources confirm code paths and configuration. The analysis does not claim successful attacks or measured denial of service.

Risk method

Score = likelihood x impact. These ordinal scores are analyst estimates, not measured probabilities. Shared causes can connect several threats.

Level	Likelihood after the stated prerequisite	Impact
1	Rare. Strong additional conditions are necessary.	Minor disruption or low-sensitivity data.
2	Unlikely. Requires control of a restricted path.	Limited disruption or a small reversible change.
3	Plausible. Requires an account, data access, or several conditions.	Material service disruption or data changes.
4	Likely. Few controls resist a capable actor.	Broad private-data disclosure or grade changes.
5	Direct path with low effort once the prerequisite holds.	Admin, OS, or DB control; broad account compromise.

High: 15-25. Medium: 6-14. Low: 1-5. Equal scores do not establish a meaningful difference in risk.

Top three

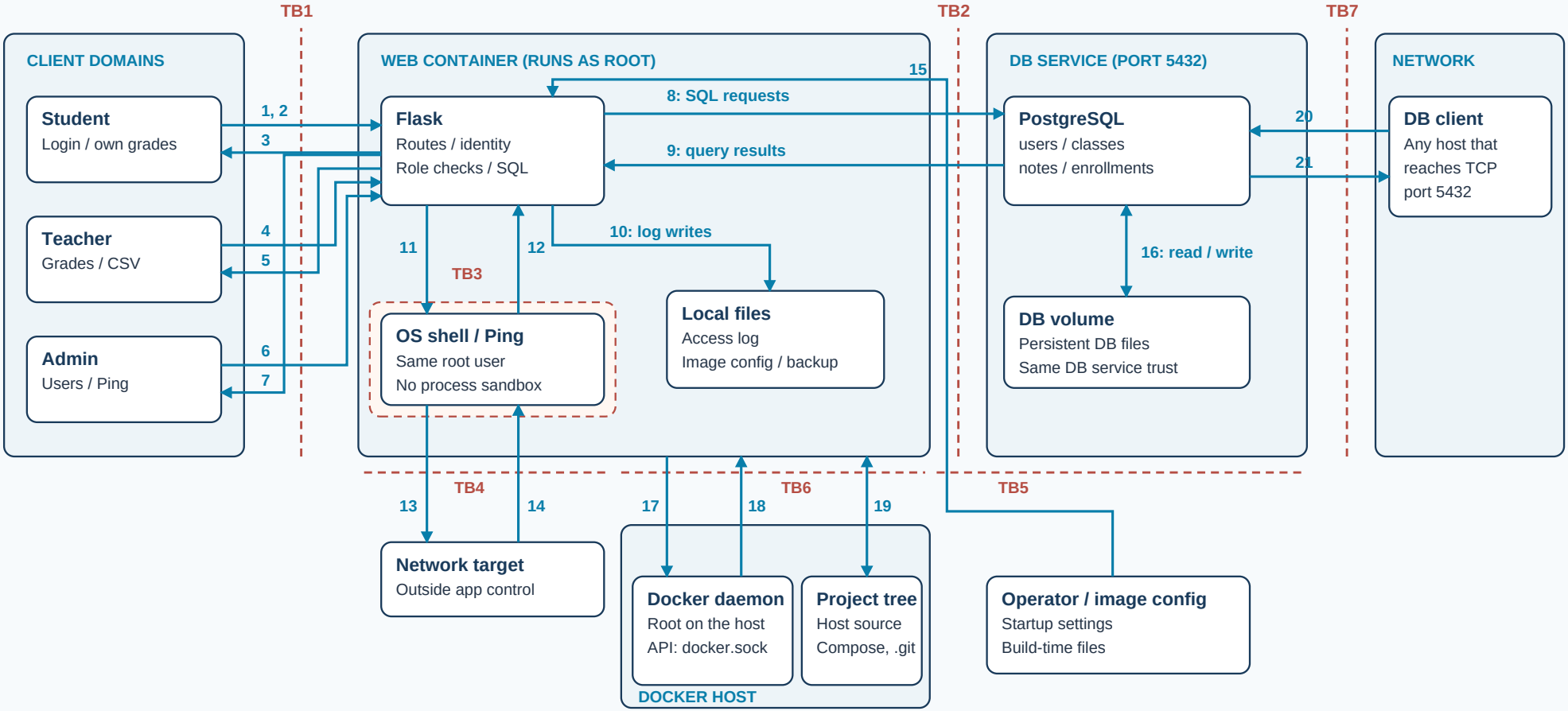
R01: forged identities, score 25. R02: login SQL injection, score 25. R15: direct DB login through the published port, score 25.

The three tie. Each one bypasses the identity or access checks of the application. R03, missing record scope, follows with 20.

Data flow and trust boundaries

NotesApp data-flow diagram

Baseline: the teacher's web/app.py and docker-compose.yml. IDs 1-9 retain the Session 02 flow numbers.



Trust boundaries

TB1: browser to web (port 8080 on all host interfaces). TB2: web to DB. TB3: input to OS execution. TB4: Ping to an external target.
TB5: operator to workload configuration. TB6: root web container to the Docker host (socket and project-tree mounts). TB7: network to the published DB port.
A boundary marks a trust change, not an effective defense. Logs and DB files remain inside their service trust.

Numbered flow register

The diagram groups related browser traffic. The register states the data and direction of each numbered flow.

ID	Sender -> receiver	Data / boundary
1	Student browser -> Flask	Login fields. TB1
2	Student browser -> Flask	Session or Bearer JWT; grade ID. TB1
3	Flask -> Student browser	Session cookie; HTML; grade JSON. TB1
4	Teacher browser -> Flask	Login; class reads; grade writes; CSV request. TB1
5	Flask -> Teacher browser	Session; classes; grades; CSV. TB1
6	Admin browser -> Flask	Login; user actions; Ping host. TB1
7	Flask -> Admin browser	Session; users; classes; Ping output. TB1
8	Flask -> PostgreSQL	SQL reads and writes; DB credentials at connection. TB2
9	PostgreSQL -> Flask	Identity; role; grades; comments; query results. TB2
10	Flask -> Local log file	Login status; username; grade author; Ping host. Local storage; no separate enforced boundary
11	Flask -> OS shell	Ping command containing the host input. TB3
12	OS shell -> Flask	Return status; stdout; stderr. TB3
13	OS Ping process -> Network target	ICMP requests; DNS lookup if a host name is used. TB4
14	Network target -> OS Ping process	ICMP or lookup responses. TB4
15	Operator and image configuration -> Flask and local files	Startup settings; application image; demo config and backup. TB5
16	PostgreSQL -> DB volume	Persistent rows and database files; read and write. DB storage; same DB service trust
17	Web container processes -> Docker daemon on the host	Docker API requests through the mounted socket; not used by app code. TB6
18	Docker daemon on the host -> Web container processes	Docker API responses; containers that the requests start. TB6
19	Web container processes -> Host project tree	Read and write host source; Compose files; db/init.sql; .git. TB6
20	Network DB client -> PostgreSQL	DB login with the published password; SQL. TB7
21	PostgreSQL -> Network DB client	Users with plaintext passwords; grades; comments; query results. TB7

Flows 13 and 14 are inferred from the Ping code. No packet capture or external Ping test was performed.

Flows 17-19 exist because of the Compose mounts. The application code does not use them; any process in the root container can. Flows 20-21 come from any client that reaches the published DB port.

Local logs and DB files share their service trust. TB3 marks command execution; the baseline provides no process sandbox.

STRIDE risk register / 1

S: Spoofing. T: Tampering. R: Repudiation. I: Information disclosure. D: Denial of service. E: Elevation of privilege. Flows are the numbered flows of the diagram.

ID / STRIDE	Flows	Scenario and prerequisite	L / I / score	Proposed fix
R01 / S/E	1-7	Forge an identity through weak token or session verification. Requires: Know the published demo configuration or send an unsigned API token.	5 / 5 / 25	Reject unsigned JWTs. Use tested verification and independent random signing keys.
R02 / T/E	1	Change the login SQL through user input. Requires: Access the login form.	5 / 5 / 25	Use parameterized SQL, password hashes, and a restricted DB role.
R03 / I/T	2, 4, 5, 8, 9	Read or change grades outside the user's scope. Requires: Have a student or teacher identity.	5 / 4 / 20	Apply student and teacher ownership checks to every affected path.
R04 / I	8, 9, 16	Read private comments from a copied database. Requires: Obtain database files or a backup copy.	3 / 4 / 12	Use AES-GCM and separate keys. Control old plaintext copies.
R05 / T/E	6, 11, 12	Execute shell commands through Admin Ping. Requires: Have an admin session or compromise it.	3 / 5 / 15	Remove shell=True. Validate the host and restrict OS privileges.
R06 / T/E	3, 5, 8, 9	Run active content through a stored comment. Requires: Write a comment that a victim later reads.	3 / 4 / 12	Enable HTML autoescape. Sanitize only approved rich text.

Evidence and confidence

R01: web/app.py:29-33;78-115;123-131. Code-supported; exploit not run.

R02: web/app.py:145-158. Code-supported; exploit not run.

R03: web/app.py:201-224;227-307. Code-supported; cross-user attack not run.

R04: db/init.sql:30-36;web/app.py:260-268. Plaintext schema confirmed; theft not simulated.

R05: web/app.py:324-340. Code-supported; command injection not run.

R06: web/templates/teacher.html:47;web/templates/dashboard.html:13-14. Code-supported; browser payload not run.

R01 has two paths. An unsigned JWT affects API identity. A forged Flask session can affect session-only teacher or admin routes.

STRIDE risk register / 2

S: Spoofing. T: Tampering. R: Repudiation. I: Information disclosure. D: Denial of service. E: Elevation of privilege. Flows are the numbered flows of the diagram.

ID / STRIDE	Flows	Scenario and prerequisite	L / I / score	Proposed fix
R07 / T	4, 6	Submit a state change from another website. Requires: The victim has a live teacher or admin browser session.	3 / 4 / 12	Check CSRF tokens on state changes. Set suitable cookie attributes.
R08 / R	4, 6, 10	Deny or alter a sensitive action record. Requires: Have access to weak logs or act through an identity that cannot be trusted.	4 / 3 / 12	Record sensitive actions and protect audit storage.
R09 / I	15, 16	Recover passwords or credentials from stored material. Requires: Read the course image, its bundled backup, or a copied user table.	3 / 5 / 15	Hash passwords. Remove bundled credential backups. Restrict secrets.
R10 / I	1-7	Read credentials or session data on an HTTP path. Requires: Control a network path between a browser and the published HTTP port.	3 / 4 / 12	Use verified HTTPS for network access.
R11 / I	8, 9, 20, 21	Read database traffic without DB TLS. Requires: Control a transport path to the DB.	2 / 4 / 8	Use verified DB TLS where required. Restrict DB connectivity.
R12 / D	1, 2, 4, 6, 8, 20	Exhaust the app or DB with repeated expensive requests. Requires: Access a permitted request path or the published DB port.	3 / 3 / 9	Limit requests, result size, database work, and execution time.

Evidence and confidence

R07: web/app.py:247-273;324-341;363-379. No CSRF check in route; browser attack not run.

R08: web/app.py:38-45;271;339;363-379. Audit gaps confirmed; log alteration not run.

R09: db/init.sql:6-12;web/Dockerfile:13-23. Storage design confirmed; no credential attack run.

R10: web/app.py:392-394;docker-compose.yml:28-29. HTTP on all host interfaces in Compose; interception not run.

R11: Runtime SHOW ssl = off;docker-compose.yml:11-12. Configuration confirmed; packet capture not run.

R12: web/app.py:48-54;227-244;276-307;docker-compose.yml has no resource limits. Availability scenario; no load measurement.

The teacher's Compose file publishes the HTTP port on all host interfaces and sets no resource limits.

STRIDE risk register / 3

S: Spoofing. T: Tampering. R: Repudiation. I: Information disclosure. D: Denial of service. E: Elevation of privilege. Flows are the numbered flows of the diagram.

ID / STRIDE	Flows	Scenario and prerequisite	L / I / score	Proposed fix
R13 / I	1, 7	Expose implementation details through errors or debug mode. Requires: Trigger an error on a reachable course route.	3 / 3 / 9	Disable debugging. Return generic errors. Protect diagnostics.
R14 / E	8, 9	Use the application DB login with excessive DB authority. Requires: Execute attacker-controlled SQL through the application, for example R02.	3 / 5 / 15	Use a DB role with only required table and sequence privileges.
R15 / S/T/I/E	20, 21	Log in to PostgreSQL directly through the published port. Requires: Reach TCP 5432 on the host; the password is in the public Compose file.	5 / 5 / 25	Close the DB port. Keep the DB on an internal network. Use a secret password and a least-privilege role.
R16 / T/I/E	11, 17, 18, 19	Take control of the Docker host through the mounted Docker socket or project tree. Requires: Run a command in the web container, for example through R05.	3 / 5 / 15	Remove the socket and project-tree mounts. Run as a non-root user. Drop capabilities.
R17 / I/D	6, 13, 14	Use Admin Ping to probe internal hosts or send traffic to other networks. Requires: Have an admin session or compromise it.	3 / 2 / 6	Allow only approved targets. Block internal ranges. Rate-limit Ping.
R18 / S/T	14	Return false Ping results through spoofed DNS or ICMP replies. Requires: Control DNS resolution or the network path to the target.	2 / 2 / 4	Treat Ping output as a hint. Prefer IP targets. Use an authenticated health check.

Evidence and confidence

R13: web/app.py:157-160;383-394. Source confirms debug and detailed errors; debugger not tested.

R14: Runtime current_user role flags: superuser;createdb;createrole. Role flags confirmed; privileged operation not run.

R15: docker-compose.yml:5-7;11-12;25;Runtime current_user role flags: superuser. Configuration confirmed; remote DB login not attempted.

R16: docker-compose.yml:33-37;web/Dockerfile:24;web/app.py:324-340. Configuration confirmed; host access not attempted.

R17: web/app.py:324-340;docker-compose.yml has no egress restriction. Code-supported; network probe not run.

R18: web/app.py:330-341. Code-inferred; spoofing not attempted.

DB role flags were checked directly: the application login is a superuser and can create databases and roles. No privileged operation was attempted.

R15 is the direct network path to the database. R04, R09, and R14 cover copies and SQL paths through the application. No DB login, host access, or network probe was attempted.

Fix the top three risks

These are proposed fixes for the original course baseline.

1. R01 / Trust verified identities

Reject alg:none and unexpected token algorithms. Use a maintained token verifier with required claim checks and expiry.

Generate independent random signing keys for tokens and Flask sessions. Store them outside source code. Invalidate sessions signed with old demo keys.

Acceptance check: reject unsigned or altered tokens. Reject old forged sessions. Confirm that valid users retain their permitted access.

2. R02 / Separate input from SQL

Use SQL parameters for the login fields. Verify a password hash instead of comparing a stored plaintext password.

Give the application DB login only the required table and sequence permissions. Remove superuser and role-management authority.

Acceptance check: login input cannot change SQL structure. Valid logins work. Invalid credentials fail. The app role cannot manage DB roles.

3. R15 / Close the database to the network

Remove the published 5432 port. Keep PostgreSQL on an internal Docker network. Keep the DB password outside the repository. Give the app a non-superuser role.

Acceptance check: a client outside the Docker network cannot open TCP 5432. The old published password fails. The app still reads and writes grades.

Next: R03, missing record scope (20). Check student note ownership and teacher class ownership on every read, write, page, and export.

Presentation and files

The three-minute speaking outline is in project1-presentation-3min.md. Editable evidence is in project1-flows.csv, project1-risks.csv, and img/project1-data-flow.svg.

Source: Session 03 Symmetric Encryption.pdf, slide 31. STRIDE definitions: https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats